

Cross-Traffic Intersection Management for Autonomous Vehicles

Week 4 — Proposal for HW/SW Codesign

Prof. Dr. Achim Rettberg · Prof. Dr. Stefan Henkler
Team A · Group 01

1 Objective of Week 4

The course plan splits the project into weekly milestones. Week 3 delivered the first abstract implementation, Week 4 asks for a proposal for the hardware/software codesign, and Week 5 is the codesign implementation itself in FreeRTOS and ModelSim. This document is the Week 4 proposal. It describes how we plan to split the RSU intersection controller between software and hardware for Week 5.

The proposal is built on two things we already have: the UPPAAL model that was verified in Week 2 (`RSU_MM1_MM2_priority_emergency_intersection`) and the deterministic C core written in Week 3. Since both already exist and agree with each other, the partition does not change the behaviour or the safety contract. It only decides where each part runs.

The rest of the document recaps the specification, looks at the Week 3 code to decide what is worth moving into hardware, describes the proposed software and hardware parts together with the interface between them, adds the fairness term that the Week 3 results showed we need, and finishes with a verification plan and a task list for Week 5.

2 Recap of Weeks 1–3 (what is being partitioned)

Week 1 defined the functional requirements. The RSU has to serialise conflicting movements, give priority to emergency and main-street traffic, support both M/M/1 and M/M/2 service, and use only V2I, I2V and V2V messages. Week 2 turned this into a UPPAAL network of timed automata with two templates, `Vehicle` and `RSU_Controller`, instantiated as one RSU and six vehicles. Six queries were checked and all of them hold. Week 3 implemented the same model as a single-threaded, deterministic C program and re-checked the safety invariants while it runs.

The fleet, the priority order and the conflict rules stay exactly as they were verified. The main street runs EAST–WEST and the side street NORTH–SOUTH, with the RSU owning the conflict zone. M/M/1 lets one vehicle cross at a time, while M/M/2 lets two cross if they do not conflict, selected by `MM_MODE`. Priority goes to the emergency vehicle first, then to main-street traffic, then to the side street, and a right turn is taken before a left turn inside a conflict. The critical pair is V4 (North-right) and V5 (South-left), which must never cross together. In UPPAAL this is the query `A[] not (crossing[4] && crossing[5])`.

Week 3 also showed one problem that this proposal has to deal with. With the strict static priority and continuous arrivals, V5 is never served, even when the simulation is extended to 400 ticks. The UPPAAL queries only check safety and reachability, not liveness, so nothing in the model forbids this. We address it with the aging term in Section 8.

3 Codesign Methodology and Partitioning Rationale

We use the standard codesign approach from the module. First we look at the reference implementation, then we sort the functions by how they compute, and then we put the regular, parallel, timing-critical work in hardware while keeping the irregular, control-heavy and often-changed code in software. When this is applied to the Week 3 functions, the split is fairly clear.

3.1 Parts that stay in software

The scheduler loop, the message handling for V2I/I2V/V2V, and the state-update functions `requestVehicle()`, `grantVehicle()` and `finishVehicle()` are sequential and full of branches, and they depend on the per-vehicle timers for period and crossTime. The priority functions `priorityValue()`, `betterPriority()`, `existsBetterGrantable()` and `emergencyWaiting()` decide the order of service and are the parts we are most likely to change later, for example when we add aging. All of this fits well as FreeRTOS tasks and queues, so it stays in software.

3.2 Parts that move to hardware

The conflict checking is different. `conflict(a,b)`, `conflictsWithCrossing(i)` and `noCollision()` are pure functions of the current `origin[]`, `turn[]`, `emergency[]` and `crossing[]` values. They have no timers and keep no state of their own. In software they are also the most repeated work on the decision path:

- `noCollision()` is a double loop, $O(N^2)$, which is 15 pairwise `conflict()` checks for $N = 6$, and it runs on every tick as the safety monitor.
- `conflictsWithCrossing(i)` is an $O(N)$ scan, and `capacityAvailableFor(i)` calls it for every candidate the RSU looks at during a grant.

In hardware these 15 pairwise checks all happen in parallel in a single clock cycle, so the timing is constant and easy to bound, and the CPU no longer spends time on them. The same combinational block can keep running every clock against the live `crossing[]` set, which gives us an independent hardware check of the `A[] noCollision()` invariant. That is the main reason we want this part in hardware: it helps both the timing and the safety. Figure 1 shows the partition, and Figure 2 shows why this kernel dominates a single grant decision.

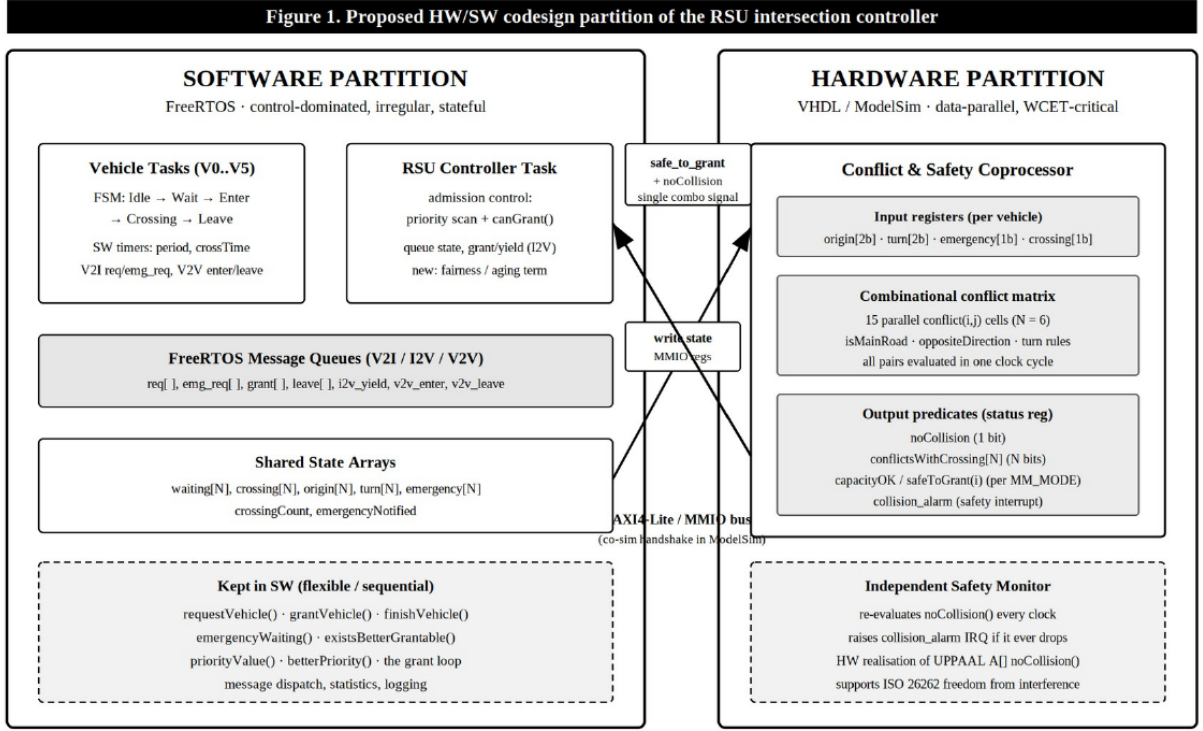


Figure 1: Proposed HW/SW codesign partition. Software (FreeRTOS) keeps the admission and scheduling logic and the messages; hardware (VHDL/ModelSim) holds the Conflict & Safety Coprocessor, which gives the software one *safe_to_grant* signal and an independent collision monitor.

4 Proposed System Architecture

4.1 Software part (FreeRTOS)

- Vehicle tasks (V0–V5): one task each, running the Vehicle FSM from UPPAAL (Idle, WaitingInQueue, V2V_Enter, Crossing, V2V_Leave). The clock $x[i]$ becomes a FreeRTOS timer or `vTaskDelay` for period and crossTime, and the committed enter and leave steps are zero-delay transitions.
- RSU controller task: takes the requests, runs the priority scan and the grant loop, sends the grants and the emergency yield, and holds the queue state. Instead of computing the conflicts itself, it asks the hardware.
- Communication: the channels `req[]`, `emg_req[]`, `grant[]`, `leave[]`, `i2v_yield`, `v2v_enter` and `v2v_leave` become FreeRTOS message queues, with the broadcast channels handled as one-to-many notifications.
- Shared state: `waiting[N]`, `crossing[N]`, `origin[N]`, `turn[N]`, `emergency[N]`, `crossingCount` and `emergencyNotified`, guarded by a mutex and copied into the coprocessor input registers before a decision.

4.2 Hardware part (VHDL / ModelSim): the Conflict & Safety Coprocessor

This is a combinational module with a small register interface that works out the conflict relation for the whole fleet at once.

- Inputs (registered): origin (2 bits), turn (2 bits), emergency (1 bit) and crossing (1 bit) per vehicle, so 6 bits times N .
- Conflict matrix: 15 `conflict(i,j)` cells in parallel, each one a small Boolean expression for `isMainRoad`, `oppositeDirection`, the same-origin case and the turn rules. This is a direct translation of the C `conflict()` function into logic.
- Outputs (status register): `noCollision` (1 bit), `conflictsWithCrossing[N]` (N bits), `capacityOK` / `safe_to_grant(i)` for the active `MM_MODE`, and `collision_alarm`.
- Safety monitor: the same matrix is evaluated every clock against the current `crossing[]` set. If `noCollision` ever goes low it raises `collision_alarm` as an interrupt. This is an independent safety check, and it fits the idea of freedom from interference in ISO 26262.

5 The `safe_to_grant` Signal

In Week 3, `canGrant(i)` checks four things in a row. For the codesign we keep `canGrant()` in software but move only the conflict-related check into hardware, so the function still reads clearly:

```
bool canGrant(int i) {
    if (!waiting[i]) return false;           // SW: queue state
    if (emergencyWaiting() && emergency[i]==0) return false; // SW: emergency hold
    if (!hw_safe_to_grant(i)) return false;  // HW: capacity + conflict (1
    cycle)
    if (existsBetterGrantable(i)) return false; // SW: strict priority
    return true;
}
```

Here `hw_safe_to_grant(i)` replaces the Week 3 `capacityAvailableFor(i)`, which used to call `conflictsWithCrossing(i)`. In M/M/1 it is true only when `crossingCount` is 0. In M/M/2 it is true when fewer than two vehicles are crossing and candidate i does not conflict with any of them, and that conflict part is what the hardware computes in parallel. The result is one bit, which removes the most expensive step of the decision and turns it into a single register read.

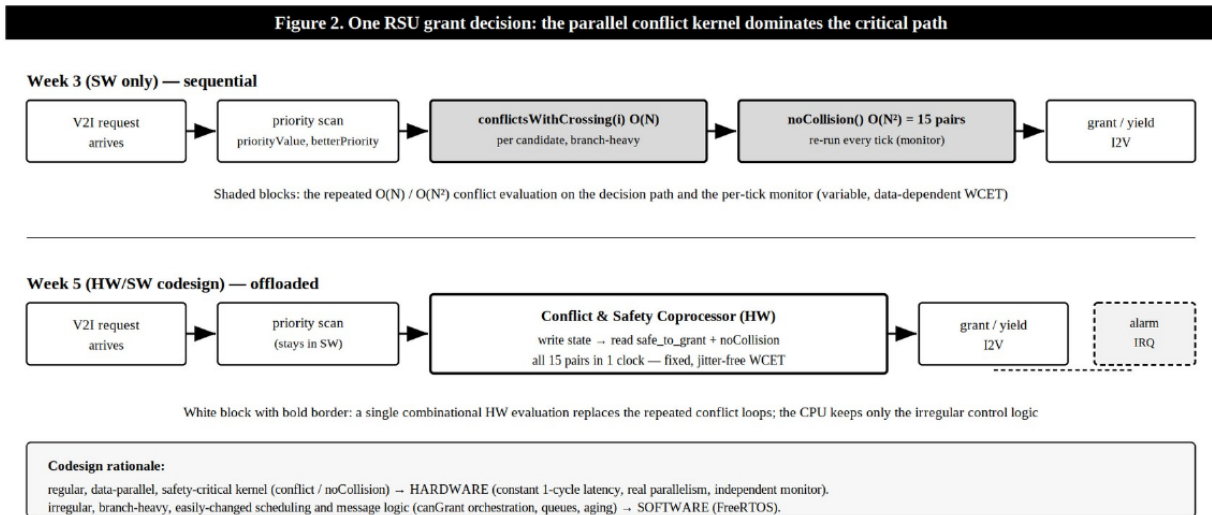


Figure 2: One grant decision. The repeated $O(N)$ and $O(N^2)$ conflict evaluation from Week 3 (shaded) becomes a single combinational hardware step in Week 5; the software keeps only the irregular control logic.

6 HW/SW Interface

Software and hardware talk to each other through a small set of memory-mapped registers. On a real Zynq or Kria board this would be an AXI4-Lite slave; in the ModelSim co-simulation it is a simple register bus. The idea is kept deliberately small: write the state, start the evaluation, read the results.

Register	Dir	Width	Meaning
STATE_VEH[i]	W	6 bits	origin(2), turn(2), emergency(1), crossing(1) for vehicle i
MODE	W	1 bit	MM_MODE: 0 = M/M/1, 1 = M/M/2
CAND	W	3 bits	candidate id to test for safe_to_grant
CTRL	W	1 bit	start / latch inputs (result valid in the same access)
ST_SAFE	R	1 bit	safe_to_grant(CAND) for the current state and MODE
ST_NOCOL	R	1 bit	noCollision() over the whole fleet
ST_CWC	R	N bits	conflictsWithCrossing[i] bit-vector
ST_ALARM	R/IRQ	1 bit	collision_alarm (latched safety violation)

Table 1: Memory-mapped register interface between the FreeRTOS RSU task and the hardware coprocessor.

For each decision the RSU task writes the current `STATE_VEH[]` and `MODE`, and then, for each candidate, it writes `CAND` and reads `ST_SAFE`. The conflict matrix is combinational, so the answer is ready in the same bus access. `ST_ALARM` is connected to an interrupt, so a safety violation is caught on its own, separately from the scheduling code.

7 Partition Mapping (UPPAAL / C $\rightarrow$ HW or SW)

Each part of the verified model has exactly one place in the codesign, which keeps the well-defined mapping from models that the brief asks for.

UPPAAL / C construct	Partition	Reason
<code>conflict(a,b)</code>	HW	pure Boolean of static rules; 15 cells in parallel
<code>conflictsWithCrossing(i)</code>	HW	$O(N)$ scan becomes an N-bit combinational vector
<code>noCollision()</code>	HW	$O(N^2)$ monitor every tick; one cycle plus alarm IRQ
<code>capacityAvailableFor(i)</code>	HW (+SW)	capacity compare folded into <code>safe_to_grant</code>
<code>priorityValue / betterPriority</code>	SW	orchestration; changes with the aging term
<code>existsBetterGrantable(i)</code>	SW	depends on the priority policy, kept flexible
<code>emergencyWaiting()</code>	SW	trivial scan; drives the I2V yield broadcast
<code>canGrant(i)</code>	SW	thin wrapper that calls <code>hw_safe_to_grant()</code>
<code>requestVehicle / grantVehicle / finishVehicle</code>	SW	state updates plus queue and message side effects
Vehicle FSM (5 locations) + <code>clock x[i]</code>	SW	one FreeRTOS task per vehicle, with SW timers

RSU_Controller (req/grant/leave/yield)	SW	RSU task plus FreeRTOS queues
channels req/grant/leave/i2v_yield/v2v_*	SW	message queues / task notifications

Table 2: HW/SW assignment of each model and code construct, with the reason.

8 Extension: Fairness / Aging (fixing the V5 starvation)

To stop V5 from starving, we add an aging term to the priority, so that the longer a vehicle waits the higher its effective priority becomes. We keep a per-vehicle counter `age[i]` that goes up every tick while `waiting[i]` is true and is reset when the vehicle is granted, and the comparison becomes something like `priorityValue(i) - k*age[i]`. Aging is small, it keeps state, and it depends on the policy, so it belongs in software, inside `betterPriority()`. The hardware conflict check does not change.

Aging never overrides safety. Emergency exclusivity, the capacity limit and the V4/V5 mutual exclusion are still enforced by `safe_to_grant`, so aging only changes which of the safe candidates is chosen and makes sure every vehicle is eventually served. This changes the Week 3 result from safety only to bounded waiting, which we add as a new query in Section 9.

9 Verification and Co-Simulation Plan (Week 5)

The plan keeps the verified UPPAAL queries as the contract and checks both the software and the hardware against them.

- Hardware (ModelSim): a VHDL testbench feeds the coprocessor with state vectors and compares `noCollision` and `safe_to_grant` against reference values taken from the Week 3 C run. The important vector is V4 (North-right) and V5 (South-left) both crossing, which has to report a conflict.
- Exhaustive check: the input space is small (6 bits per vehicle over the few origin and turn values the fleet actually uses), so we can compare the hardware outputs with the C reference over all reachable states, not just a sample.
- Software (FreeRTOS): the FreeRTOS version has to reproduce the Week 3 trace, that is the M/M/2 overlap of V1 and V2 and the emergency case where V0 pre-empts after an I2V yield. The three safety invariants stay as run-time assertions.
- Co-simulation: the RSU task drives the coprocessor over the register interface, and the test fails if `hw_safe_to_grant` ever disagrees with the C reference.

Table 3 maps the six original queries onto the codesign and adds the new liveness query.

UPPAAL query	Where checked	Mechanism
A[] not deadlock	SW	cyclic schedule always has an enabled action
A[] noCollision()	HW + SW	HW monitor every clock, plus SW assert
A[] crossingCount <= MM_MODE	HW + SW	capacity in safe_to_grant, plus SW assert
A[] not(crossing[4]&&crossing[5])	HW + SW	conflict cell (4,5) = 1, regression vector
E<> crossing[0]	SW	emergency V0 granted (log and counter)
E<> crossing[1] && crossing[2]	SW	V1 and V2 cross together in M/M/2
waiting[5] -> crossing[5] (new)	SW	bounded waiting once aging is on

Table 3: Verification correspondence for the codesign, with the new liveness query added by the aging term.

10 Week 5 Work Plan, Deliverables and Risks

10.1 Deliverables

- SW: the FreeRTOS implementation (vehicle tasks, RSU task, queues, timers, aging) reproducing the Week 3 behaviour.
- HW: a synthesisable VHDL Conflict & Safety Coprocessor with a ModelSim testbench and the golden regression vectors.
- Integration: the register-interface driver and a co-simulation demonstration, with the results pushed to the repository.
- A report chapter that documents the partition, the interface and the verification results.

10.2 Risks and mitigations

Risk	Mitigation
HW and SW state drift (stale registers)	one writer (the RSU task) mirrors the state before each decision; a mutex guards the shared arrays
Interface overhead cancels the HW benefit	keep it combinational with a single-cycle register read; write the state once per tick
ModelSim setup or licensing	use the Intel/Quartus Prime ModelSim edition from the brief and start the testbench early
Aging breaks a safety invariant	aging only reorders safe candidates; safety is still gated by safe_to_grant; add the liveness query
Behaviour drifts from Week 3	golden vectors and assertions tie both parts back to the verified C reference

Table 4: Main risks for the Week 5 codesign implementation and how we plan to handle them.

11 Summary

The proposed split keeps the verified contract and puts each part of the Week 3 core where it fits best. The control and scheduling logic, which is irregular and likely to change, stays in software

on FreeRTOS. The conflict checking, which is regular, parallel and safety-relevant (`conflict`, `conflictsWithCrossing`, `noCollision`), moves into a hardware Conflict & Safety Coprocessor that gives the software one `safe_to_grant` signal and an independent collision monitor. This makes the timing of a grant decision constant and easy to bound, adds a separate hardware check of `A[]noCollision()`, and takes load off the CPU. The aging term fixes the V5 starvation seen in Week 3 and raises the guarantee from safety only to bounded waiting. With this in place, the design is ready to be implemented and verified in FreeRTOS and ModelSim in Week 5.